

# MACHINE LEARNING & NLP

Complete classroom-style notes for learning, revision, projects, and interviews.

This guide organizes the full training sequence discussed above: data distribution, probability, machine learning foundations, regression, model evaluation, regularization, clustering, NLP preprocessing, text vectorization, Word2Vec, RNN, LSTM, and GRU. Explanations are simplified, but important technical corrections are included so the notes remain useful for future study.

# 1. Data Distribution, Skewness and Outliers

**Skewness** measures asymmetry in a distribution. A symmetric bell-shaped distribution has approximately equal left and right sides.

| Distribution          | Tail            | Typical relation                     | Meaning                                 |
|-----------------------|-----------------|--------------------------------------|-----------------------------------------|
| Symmetric             | Balanced        | Mean $\approx$ Median $\approx$ Mode | Values are balanced around the center.  |
| Right / positive skew | Long right tail | Mode < Median < Mean                 | A few large values pull the mean right. |
| Left / negative skew  | Long left tail  | Mean < Median < Mode                 | A few small values pull the mean left.  |

**Example:** If most exam scores are 70–90 but a few scores are 1, 12 and 20, the long tail is on the left, so the distribution is left-skewed.

**Key point:** Skew direction is named after the long tail. An outlier should not automatically be deleted; first determine whether it is an error, a rare valid event, or important business information.

## 2. Probability Distributions

### Normal / Gaussian Distribution

A continuous, symmetric, bell-shaped distribution. Many measurements can be approximately normal. The mean determines the center and standard deviation determines spread.

### Bernoulli Distribution

Represents one trial with two outcomes: head/tail, pass/fail, fraud/not fraud. Bernoulli = one binary trial.

### Binomial Distribution

Counts successes across a fixed number of independent Bernoulli trials with constant success probability. Example: number of heads in 10 coin tosses.

### Uniform Distribution

All outcomes in the allowed range are equally likely. For a fair six-sided die, each face has probability 1/6.

### Poisson Distribution

Models the number of events in a fixed time or space interval when events occur independently at an approximately constant average rate. Examples: calls per hour or defects per meter.

## 3. Empirical Rule and Standardization

| Distance from mean               | Approximate normal-distribution coverage |
|----------------------------------|------------------------------------------|
| Mean $\pm$ 1 standard deviation  | 68%                                      |
| Mean $\pm$ 2 standard deviations | 95%                                      |
| Mean $\pm$ 3 standard deviations | 99.7%                                    |

The **Z-score** standardizes a value:  $z = (x - \text{mean}) / \text{standard deviation}$ . After standardization, a feature has mean approximately 0 and standard deviation approximately 1.

**Key point:** A Z-score is a standardized value, not automatically a probability.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Learn scaling parameters only from training data.
X_train_scaled = scaler.fit_transform(X_train)

# Apply the same learned transformation to test data.
X_test_scaled = scaler.transform(X_test)
```

## 4. What is Machine Learning?

Machine Learning is a field of Artificial Intelligence where algorithms learn patterns from data and use those patterns to make predictions, classifications, recommendations, or decisions on new data.

**Why do we need it?** Humans cannot repeatedly inspect millions or billions of records manually. ML helps automate pattern recognition at scale, produce consistent predictions, and adapt through retraining when data changes.

**Example:** Instead of manually checking millions of financial transactions, a fraud model learns historical patterns and scores new transactions.

## 5. Types of Machine Learning

| Type                   | Data                          | Purpose                                   | Examples                   |
|------------------------|-------------------------------|-------------------------------------------|----------------------------|
| Supervised             | Labeled                       | Learn input → target mapping              | Regression, classification |
| Unsupervised           | Unlabeled                     | Discover hidden structure                 | Clustering, PCA            |
| Semi-supervised        | Some labeled + much unlabeled | Learn with limited labels                 | Image/text tasks           |
| Reinforcement learning | Interaction + rewards         | Learn actions maximizing long-term reward | Robotics, games, control   |

**Key point:** Reinforcement learning is sequential decision-making using states, actions and rewards; it is more than simply rewarding a correct prediction.

## 6. Regression vs Classification

| Problem        | Output            | Example                              |
|----------------|-------------------|--------------------------------------|
| Regression     | Continuous number | Predict monthly rent = \$1,850       |
| Classification | Category          | Predict rent class = Low/Medium/High |

**Binary classification:** two classes. **Multiclass:** more than two classes, normally one class per sample. **Multilabel:** one sample can have multiple labels simultaneously.

**Key point:** Separate cat, dog and bird images form a multiclass problem. One image containing both a cat and dog can be multilabel.

## 7. Machine Learning Lifecycle

### Problem definition

Define business objective, target, constraints, success metric and cost of errors.

### Data collection

Gather relevant historical data.

### Data understanding

Understand columns, units, types and business meaning.

## EDA

Study distributions, relationships, outliers, missing values and imbalance.

## Cleaning and feature engineering

Fix data quality and create useful model inputs.

## Preprocessing

Encode categories and scale features when required.

## Train/validation/test strategy

Keep unseen data for honest evaluation.

## Model selection and training

Choose suitable algorithms and fit them.

## Evaluation and tuning

Use appropriate metrics and cross-validation.

## Deployment

Serve predictions through batch, streaming, applications or APIs.

## Monitoring and retraining

Track drift, performance, failures and business impact.

**Key point:** A pickle file is only one possible model serialization format. Production deployment may use joblib, ONNX, framework formats, registries, containers, or managed serving systems.

## 8. Linear Regression

Linear regression predicts a continuous target using a linear relationship. For one feature:  $y = mx + c$ . Here  $y$  is the prediction,  $x$  is the input,  $m$  is the slope/coefficient, and  $c$  is the intercept.

The goal is to find a line that produces small prediction errors across the observed data.

### Residual

**Residual = actual value – predicted value.** A residual is the error for one observation. A loss function combines errors across many observations.

### Ordinary Least Squares (OLS)

OLS chooses coefficients that minimize the sum of squared residuals:  $\sum (y - \hat{y})^2$ . A matrix form of the closed solution is  $\hat{\beta} = (X^T X)^{-1} X^T y$ .

**Key point:** The closed-form OLS solution does not repeatedly iterate. Gradient descent is the iterative alternative.

### Gradient Descent

Gradient descent starts with parameter values, calculates the direction in which the loss changes, and repeatedly updates parameters to reduce loss.

**new weight = old weight – learning rate × gradient**

A very small learning rate can be slow. A very large learning rate can overshoot or diverge.

**Key point:** For standard linear regression with mean squared error, the loss surface is convex, so the main target is one global minimum.

## 9. Regression Metrics

| Metric                  | Meaning                                      | Behavior                                          |
|-------------------------|----------------------------------------------|---------------------------------------------------|
| MAE                     | Average absolute error                       | Linear penalty; relatively robust to large errors |
| MSE                     | Average squared error                        | Large errors receive stronger penalty             |
| RMSE                    | Square root of MSE                           | Same unit as target; sensitive to large errors    |
| R <sup>2</sup>          | Variance explained relative to mean baseline | Higher generally better, context matters          |
| Adjusted R <sup>2</sup> | R <sup>2</sup> adjusted for predictor count  | Penalizes unnecessary predictors                  |

**R<sup>2</sup> = 1 – RSS/TSS.** If R<sup>2</sup> = 0.75, the model explains about 75% of target variance relative to the mean baseline. It does not mean 75% of individual predictions are correct.

## 10. Multiple Linear Regression

With several predictors:  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$ . Example: sales predicted from TV, radio and newspaper advertising.

Ordinary R<sup>2</sup> cannot decrease when a predictor is added. Adjusted R<sup>2</sup> accounts for model size and is more useful for comparing models with different numbers of predictors.

## 11. Multicollinearity

Multicollinearity occurs when predictors are strongly related to each other. This can make coefficients unstable and difficult to interpret.

**Detection:** correlation matrix, heatmap and Variance Inflation Factor (VIF). VIF above 5 often deserves investigation; above 10 can indicate serious multicollinearity, but thresholds are not universal.

**Remedies:** remove a redundant variable, combine features, use PCA, use regularization, or leave it alone when prediction is the only goal and performance is stable.

## 12. Overfitting, Underfitting, Bias and Variance

| Condition    | Training  | Unseen data | Meaning                                       |
|--------------|-----------|-------------|-----------------------------------------------|
| Underfitting | Poor      | Poor        | Model too simple or features inadequate       |
| Good fit     | Good      | Good        | Useful generalization                         |
| Overfitting  | Very good | Much worse  | Model learned training-specific noise/details |

**Bias** is systematic error caused by restrictive assumptions. **Variance** is sensitivity to changes in the training sample. High bias is associated with underfitting; high variance is associated with overfitting.

**Key point:** Class imbalance and bias in the bias–variance tradeoff are different concepts.

## 13. Regularization

Regularization adds a penalty to discourage excessively large coefficients and reduce model variance.

| Method      | Penalty                        | Main behavior                        |
|-------------|--------------------------------|--------------------------------------|
| Lasso / L1  | $\lambda \times \sum( \beta )$ | Can set coefficients exactly to zero |
| Ridge / L2  | $\lambda \times \sum(\beta^2)$ | Shrinks coefficients toward zero     |
| Elastic Net | L1 + L2                        | Combines sparsity and shrinkage      |

As  $\lambda$  increases, regularization becomes stronger. Too much regularization can make the model too simple and cause underfitting.

**Key point:** Choose Lasso, Ridge or Elastic Net using cross-validation and the problem structure—not a simplistic rule such as 'high variance means always use Lasso.'

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet

ridge = Ridge(alpha=1.0)           # L2 penalty
lasso = Lasso(alpha=0.01)          # L1 penalty
elastic = ElasticNet(alpha=0.01,   # Mix of L1 and L2
                    l1_ratio=0.5)

ridge.fit(X_train_scaled, y_train)
predictions = ridge.predict(X_test_scaled)
```

## 14. Polynomial Regression

When the relationship is curved, polynomial features such as  $x^2$  and  $x^3$  can be added. Example:  $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$ . Higher degree increases flexibility but can overfit.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

model = Pipeline([
    # Create x, x^2 and interaction terms.
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),

    # Fit linear regression on the expanded features.
    ("regression", LinearRegression())
])
```



```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

## 15. Class Imbalance

Class imbalance occurs when one class is much more common than another, such as 98% legitimate transactions and 2% fraud. Accuracy alone can be misleading.

Use confusion matrix, precision, recall, F1, PR-AUC/ROC-AUC where appropriate, class weights, careful oversampling/undersampling, and threshold tuning based on business costs.

## 16. Unsupervised Learning and Clustering

Unsupervised learning uses unlabeled data to discover hidden structure. Common uses include customer segmentation, document grouping, exploratory anomaly detection and dimensionality reduction.

### Agglomerative vs Divisive

| Approach      | Start                         | Process                                      |
|---------------|-------------------------------|----------------------------------------------|
| Agglomerative | Each point is its own cluster | Repeatedly merge similar clusters; bottom-up |
| Divisive      | One large cluster             | Repeatedly split; top-down                   |

## 17. K-Means Clustering

K-Means partitions observations into K groups using centroids.

**Steps:** choose K → initialize centroids → assign each point to nearest centroid → recompute centroids as means → repeat until stable.

**Choosing K:** elbow method, silhouette score and domain usefulness.

**Key point:** K-Means is distance-based, sensitive to scale, initialization and outliers, and works best for relatively compact clusters.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X_scaled = StandardScaler().fit_transform(X)

model = KMeans(
    n_clusters=3,
    random_state=42,
    n_init="auto"
)

# Fit the model and return a cluster ID for each row.
labels = model.fit_predict(X_scaled)
```

## 18. Hierarchical Clustering

Hierarchical clustering builds nested groups. A **dendrogram** shows how observations or clusters merge. Cutting the dendrogram at a chosen height produces flat clusters.

Common linkage methods include single, complete, average and Ward linkage.

**Key point:** The cluster count is not chosen merely by finding the smallest number of lines touched by a horizontal cut. Large merge-distance gaps, metrics and domain usefulness should all be considered.

## 19. DBSCAN

DBSCAN is Density-Based Spatial Clustering of Applications with Noise. It finds dense regions and can label isolated observations as noise.

| Term | Meaning             |
|------|---------------------|
| eps  | Neighborhood radius |

| Term         | Meaning                                   |
|--------------|-------------------------------------------|
| min_samples  | Minimum local density requirement         |
| Core point   | Has enough neighbors within eps           |
| Border point | Not core, but reachable from a core point |
| Noise point  | Neither core nor reachable as border      |

DBSCAN does not require K in advance and can find irregular cluster shapes, but selecting eps and min\_samples is important.

## 20. Silhouette Score

For a point  $i$ ,  $a(i)$  is average distance to its own cluster and  $b(i)$  is the smallest average distance to another cluster. Scores near +1 are good, near 0 indicate a boundary, and negative values suggest possible misassignment.

**Key point:** A high silhouette score does not automatically mean the clusters are useful for the business.

## 21. Natural Language Processing (NLP)

NLP is a field of AI focused on computational processing, analysis, understanding and generation of human language.

**Applications:** search, chatbots, translation, sentiment analysis, spam detection, information extraction, summarization and question answering.

## 22. Tokenization

Tokenization breaks text into smaller units. Sentence tokenization splits paragraphs into sentences. Word/subword tokenization splits text into tokens.

**Key point:** Modern transformer models commonly use subword tokenization rather than simple word splitting.

## 23. Stop Words

Stop words are frequent function words such as 'the', 'is' and 'and'. Removing them can help some classical NLP pipelines reduce vocabulary size.

**Key point:** Do not remove stop words blindly. Words such as 'not' can completely change meaning, and transformer models usually retain function words.

## 24. POS Tagging

Part-of-Speech tagging labels words by grammatical role: noun, verb, adjective, adverb, and so on. This provides linguistic information about how a token is being used.

## 25. Stemming vs Lemmatization

| Stemming             | Lemmatization                                      |
|----------------------|----------------------------------------------------|
| Rule-based chopping  | Linguistically informed normalization              |
| May create non-words | Usually returns a valid dictionary lemma           |
| Faster and rougher   | More context-sensitive and computationally heavier |

**Example:** stemming may reduce 'studies' to 'studi'. Lemmatization aims for a valid base such as 'study'.

## 26. Named Entity Recognition (NER)

NER identifies spans referring to entities such as people, organizations, locations, dates, money and products.

**Example:** 'Google opened an office in London in 2024' → Google: organization, London: location, 2024: date.

**Key point:** Modern NER is not just hard-coded dictionary tagging. Statistical and neural models use surrounding context to disambiguate meanings.

## 27. BIO Tagging

| Tag   | Meaning                             |
|-------|-------------------------------------|
| B-ORG | Beginning of an organization entity |

| Tag   | Meaning                                       |
|-------|-----------------------------------------------|
| I-ORG | Inside/continuation of an organization entity |
| B-PER | Beginning of a person entity                  |
| I-PER | Continuation of a person entity               |
| O     | Outside any target entity                     |

**Key point:** B means beginning of an entity, I means inside an entity, and O means outside—not beginning of a sentence and not output.

## 28. Why Text Vectorization?

Machine-learning models operate on numerical representations. Vectorization converts text into numbers. Different methods preserve different amounts of frequency, order, similarity and context.

## 29. One-Hot Encoding

Each vocabulary item gets a vector with one 1 and all remaining values 0. It is simple but sparse, high-dimensional and does not represent semantic similarity.

```
Alice    -> [1, 0, 0]
Bob      -> [0, 1, 0]
Charlie  -> [0, 0, 1]
```

## 30. Bag of Words

Bag of Words represents a document using word counts. It captures occurrence and frequency but ignores order. Therefore 'dog bites man' and 'man bites dog' can have identical counts.

## 31. TF-IDF

**Term Frequency** measures how often a word occurs in a document. **Inverse Document Frequency** lowers the weight of words appearing in many documents. **TF-IDF = TF × IDF**.

**Key point:** IDF gives lower weight to corpus-wide common terms and relatively higher weight to rarer, potentially discriminative terms.

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "machine learning learns patterns",
    "natural language processing handles text"
]

vectorizer = TfidfVectorizer()

# Learn vocabulary and convert documents to TF-IDF vectors.
X = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

## 32. N-Grams

| Type    | Example from 'I love machine learning'   |
|---------|------------------------------------------|
| Unigram | I   love   machine   learning            |
| Bigram  | I love   love machine   machine learning |
| Trigram | I love machine   love machine learning   |

N-grams preserve short-range order but vocabulary size grows quickly, rare combinations become sparse, and long-range context remains difficult.

## 33. Word Embeddings and Word2Vec

Word embeddings are dense learned vectors where semantically related words can occupy nearby regions of vector space.

**CBOW:** surrounding context → predict target word. **Skip-Gram:** target word → predict surrounding context.

**Key point:** Classic Word2Vec embeddings are static. The word 'bank' gets one vector even when context changes between a financial bank and river bank.

## 34. Recurrent Neural Network (RNN)

RNNs are designed for sequential data. At time  $t$ , the network processes the current input together with the previous hidden state:  $h_t = f(x_t, h_{(t-1)})$ .

The hidden state carries information forward through the sequence. The same RNN parameters are reused at every time step.

**Key point:** An unrolled RNN is drawn as many repeated cells, but it is not a completely new independent network for every word.

### Main limitations

**Vanishing gradients:** learning signals can become extremely small across many time steps, making long-range dependencies difficult to learn.

**Exploding gradients:** gradients can become very large and destabilize training.

**Sequential computation:** recurrent steps limit parallel processing.

**Key point:** The main RNN problem is not simply that the architecture grows and therefore overfits. The central technical difficulty is learning long-range dependencies through repeated recurrent steps.

## 35. LSTM — Long Short-Term Memory

LSTM is a gated recurrent architecture designed to preserve useful information over longer sequences and improve gradient flow.

It maintains a **cell state** for long-term memory and a **hidden state** for the current representation.

| Gate        | Purpose                                                       |
|-------------|---------------------------------------------------------------|
| Forget gate | Controls how much previous cell-state information is retained |
| Input gate  | Controls how much new candidate information is written        |
| Output gate | Controls what information contributes to the new hidden state |

**Key point:** The output gate does not directly 'choose the next word.' It controls the hidden-state output; a later output layer can use that state to predict a word or class.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

model = Sequential()

# Convert token IDs into learned dense vectors.
model.add(Embedding(input_dim=10000, output_dim=128))

# Learn sequential patterns with gated long-term memory.
model.add(LSTM(64))

# Example: binary classification output.
model.add(Dense(1, activation="sigmoid"))

model.summary()
```

## 36. GRU — Gated Recurrent Unit



GRU is a simpler gated recurrent architecture. It normally uses one hidden state rather than separate hidden and cell states.

| Gate        | Purpose                                                                         |
|-------------|---------------------------------------------------------------------------------|
| Update gate | Controls how much previous information is retained versus updated               |
| Reset gate  | Controls how much previous information contributes to new candidate information |

GRUs often have fewer parameters and may train faster than comparable LSTMs.

**Key point:** GRU is not universally an upgraded or better LSTM. Which performs better depends on the task, data, sequence length, model size and training setup.

## 37. RNN vs LSTM vs GRU

| Feature                 | RNN          | LSTM                  | GRU                       |
|-------------------------|--------------|-----------------------|---------------------------|
| Memory                  | Hidden state | Hidden + cell state   | Hidden state              |
| Gates                   | None         | Input, forget, output | Update, reset             |
| Long-range dependencies | Weakest      | Strong                | Strong                    |
| Parameters              | Fewest       | Usually most          | Usually fewer than LSTM   |
| Typical tradeoff        | Simple       | Powerful but heavier  | Simpler gated alternative |

## 38. Evolution Toward Generative AI

**Classical preprocessing** → **Bag of Words / TF-IDF** → **Word embeddings** → **RNN** → **LSTM / GRU** → **Attention** → **Transformers** → **Large Language Models**

Modern large language models are primarily transformer-based. Transformers use attention mechanisms to model relationships among tokens and allow much more parallel training than recurrent architectures.

**Key point:** AI is the broad field. Machine learning is a major subset of AI; deep learning is a subset of machine learning. NLP is a field that can use ML and deep learning. Generative AI overlaps with these areas rather than being a completely independent box.

## 39. Quick Interview Revision

### What is machine learning?

Algorithms learn patterns from data and use them to make predictions or decisions on new data.

### Regression vs classification?

Regression predicts continuous numbers; classification predicts categories.

### What is a residual?

Actual target minus predicted target for one observation.

### What is overfitting?

Excellent training fit but poor generalization to unseen data.

### What is regularization?

A penalty that discourages excessive model complexity and can improve generalization.

### Lasso vs Ridge?

Lasso uses L1 and can set coefficients to zero; Ridge uses L2 and usually shrinks without exact zeros.

### What is K-Means?

A centroid-based clustering algorithm that alternates assignment and centroid updates.

### What is DBSCAN?

A density-based clustering algorithm that can discover irregular clusters and identify noise.

### What is TF-IDF?

A text weighting method combining within-document term frequency and across-corpus inverse document frequency.

### What is RNN?

A recurrent sequence model that updates a hidden state using current input and previous state.

### Why LSTM?

To learn longer dependencies using gated memory more effectively than a vanilla RNN.

## LSTM vs GRU?

LSTM uses a cell state and three main gates; GRU uses update/reset gates and a simpler recurrent structure.

## 40. Practical Rules

1. Start with the business problem, not the algorithm.
2. Understand data before training.
3. Prevent data leakage.
4. Keep evaluation data unseen.
5. Use metrics that reflect the real cost of mistakes.
6. Do not remove outliers automatically.
7. Do not trust accuracy alone for imbalanced data.
8. Do not interpret  $R^2$  as percentage of predictions that are correct.
9. Compare models with cross-validation.
10. Monitor deployed models because real-world data changes.

## Final Learning Path

| Stage | Focus                                   |
|-------|-----------------------------------------|
| 1     | Python, NumPy, Pandas                   |
| 2     | Statistics and probability              |
| 3     | EDA and visualization                   |
| 4     | Preprocessing and feature engineering   |
| 5     | Regression and classification           |
| 6     | Trees, ensembles and model tuning       |
| 7     | Clustering and dimensionality reduction |
| 8     | NLP preprocessing and vectorization     |
| 9     | Embeddings and sequence models          |
| 10    | Attention and transformers              |
| 11    | LLMs, RAG, agents and production GenAI  |

**Final takeaway:** Machine learning is not simply loading data and calling `model.fit()`. Strong work requires a clear problem, reliable data, careful EDA, correct validation, appropriate metrics, and an understanding of each model's assumptions and limitations.